

KNeighborsClassifier (KNN 분류)

사이킷런의 대표적인 거리 기반 분류 알고리즘

1. KNN(K-Nearest Neighbors) 분류란?

KNN은 새로운 데이터가 들어왔을 때, 학습 데이터 중에서 **가장 가까운 K개 이웃**을 찾아 그 이웃들의 클래스를 기준으로 **다수결**로 분류하는 알고리즘입니다.

핵심 아이디어

“가까운 데이터는 같은 범주에 속한다”

2. fit()과 predict()의 역할

① fit() : 학습 단계

fit(X, y) 단계에서는 **모델이 규칙을 계산하지 않습니다**.
KNN은 학습 데이터 **X**와 정답 **y**를 그대로 저장하는 방식입니다.

KNN은 **지연 학습(Lazy Learning)** 알고리즘입니다.
학습 시점이 아니라 **예측 시점**에 계산이 집중됩니다.

② predict() : 예측 단계 (거리 계산)

predict()가 호출되는 순간, 새로운 데이터와 기존 데이터 간의 거리를 계산합니다. 사이킷런의 **p** 매개변수 설정에 따라 거리 계산 방식이 달라집니다.

설정 (p)	거리 방식	공식	특징
p = 2 (기본값)	유클리드 (Euclidean)	$d = \sqrt{\sum (x_i - y_i)^2}$	두 지점 사이의 최단 직선 거리
p = 1	맨해튼 (Manhattan)	$d = \sum x_i - y_i $	도로를 따라가듯 직각으로 이동하는 거리

두 방식 모두 **민코프스키(Minkowski) 거리**라는 일반 공식의 특수 형태입니다. 계산된 거리 기준으로 **가장 가까운 K개**를 선택하고, **다수결**로 최종 클래스를 결정합니다.

3. 기본 사용 예제

```
from sklearn.neighbors import KNeighborsClassifier

# 학습 데이터 (특성 1개라도 X는 2차원)
X = [[1], [2], [3], [6], [7], [8]]
y = [0, 0, 0, 1, 1, 1]

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)          # 데이터 저장 (거리 계산 ❌)

print(model.predict([[4]])) # 여기서 유클리드 거리 계산 ⓪
```

중요 포인트

- ✓ `fit()` → 데이터 저장만 수행
- ✓ `predict()` → 유클리드 거리 계산 + 다수결

4. 왜 스케일링이 중요한가요?

KNN은 유클리드 거리를 사용하므로 특성의 단위가 다르면 거리 계산이 왜곡될 수 있습니다.

- 키(cm) + 몸무게(kg)
- 나이(년) + 소득(원)

👉 따라서 KNN에서는 **표준화(StandardScaler)**나 **정규화**를 함께 사용하는 것이 매우 중요합니다.

5. 한 줄 요약

KNeighborsClassifier는 **predict()** 단계에서 유클리드 거리로 이웃을 계산하는 거리 기반 분류 알고리즘입니다.